Talk & Travel

Travel Insurance Management System

System Documentation

1. Project Overview

1.1 Purpose of the System

Talk & Travel is a full-stack web application designed to digitize and streamline the end-to-end process of purchasing and managing travel insurance. The system provides a structured, agent-verified policy workflow that ensures customers are properly vetted before policies are issued. It supports four distinct user roles — Customer, Agent, Admin, and Claims Officer — each operating within a dedicated portal with role-specific capabilities.

1.2 Problem the System Solves

Traditional and unstructured travel insurance platforms face several critical challenges:

- Customers can purchase policies without document or identity verification, creating opportunities for fraud.
- No agent oversight exists before a policy is issued, reducing quality control.
- There is no centralized portal where multiple stakeholder roles can manage their responsibilities.
- Claim approvals follow slow, unstructured, and often manual processes.
- Customers have no real-time visibility into the status of their claims.
- Premium calculation is prone to human error when performed manually.

Talk & Travel addresses each of these issues through automated agent assignment, risk scoring, structured claim lifecycles, role-based access control, and a formula-driven premium engine.

1.3 High-Level System Architecture

The application follows a client-server architecture. The Angular frontend communicates with the ASP.NET Core Web API backend via RESTful HTTP endpoints secured by JWT Bearer tokens. The backend is organized using Clean Architecture principles across four logical layers: Domain, Application, Infrastructure, and API. SQL Server is used as the relational database, accessed through Entity Framework Core. Static files such as uploaded documents are served directly from the web server file system.

1.4 Technology Stack

Layer	Technology
Backend Framework	ASP.NET Core Web API (.NET 8)
Architecture Pattern	Clean Architecture — Domain / Application / Infrastructure / API
ORM	Entity Framework Core
Database	Microsoft SQL Server
Authentication	ASP.NET Identity + JWT Bearer Tokens

Frontend Framework	Angular (Standalone Components) with Angular Signals
UI Styling	TailwindCSS
Testing	xUnit + Moq + EF Core InMemory (4 test projects)

2. System Architecture

2.1 Backend — Clean Architecture

The backend is structured using Clean Architecture, which enforces strict separation of concerns across four layers. Each layer depends only on the layer below it, preventing circular dependencies and keeping the codebase testable and maintainable.

Layer 1 — Domain

The Domain layer is the innermost layer and has no external dependencies. It contains:

- Core entity classes: ApplicationUser, PolicyProduct, PolicyRequest, Policy, Claim, and related document entities.
- Enumerations for status values such as PolicyStatus, ClaimStatus, and PolicyProductStatus.
- No business logic or framework references — pure plain C# classes.

Layer 2 — Application

The Application layer defines business rules and orchestration. It contains:

- Service interfaces (e.g., IPolicyService, IClaimService, IAgentService) that define all business operations.
- Data Transfer Objects (DTOs) used for both incoming requests and outgoing responses.
- Premium calculation logic, risk score computation, and auto-assignment rules.
- Repository interfaces (e.g., IPolicyRepository, IClaimRepository) that abstract data access.

Layer 3 — Infrastructure

The Infrastructure layer implements all interfaces defined in the Application layer:

- Concrete service implementations using Entity Framework Core.
- AppDbContext and EF Core migrations.
- ASP.NET Identity configuration for user management and password hashing.
- File system operations for document upload and retrieval.
- Data seeding for the Admin account, Agents, Claims Officer, and 11 insurance plan products.

Layer 4 — API

The API layer is the outermost layer and handles HTTP communication:

- ASP.NET Core Controllers for each role: AuthController, PolicyRequestController, PolicyController, AgentController, ClaimsOfficerController, AdminController.
- JWT Bearer token validation middleware.
- Role-based [Authorize] attributes on all protected endpoints.

- Static file serving configuration for uploaded documents (wwwroot/).
- CORS policy configured for the Angular development server.

2.2 Frontend — Angular Architecture

The frontend is built with Angular using standalone components, eliminating the need for NgModules. Angular Signals are used for reactive state management. TailwindCSS provides utility-first styling throughout the application.

- Each role has a dedicated portal with its own route group and layout.
- Angular Auth Guards protect all routes based on the decoded JWT role claim.
- A No-Auth Guard prevents already-authenticated users from accessing login and registration pages.
- HTTP Interceptors attach the JWT Bearer token to every outbound API request.
- The browse plans page polls for premium calculations every time trip parameters change.
- The My Requests page polls every 30 seconds to refresh policy request status without requiring a page reload.

2.3 Frontend-Backend Communication

All communication between the Angular frontend and the ASP.NET Core backend occurs over HTTP using JSON payloads. File uploads use multipart/form-data. JWT tokens are stored client-side and included in the Authorization header as Bearer tokens on every API request. The backend validates the token signature, expiry, and role on every protected endpoint before processing the request.

3. User Roles and Responsibilities

The system supports four distinct user roles. Each role has access only to its own portal and endpoints. Role assignment is fixed at the time of account creation or seeding.

Role	Responsibilities
Customer	Self-registers; browses and filters insurance plans; submits policy requests with KYC documents; pays for approved requests; submits claims with supporting evidence; tracks request and claim statuses in real time.
Agent	Reviews assigned policy requests; views risk scores and factor breakdowns; approves or rejects requests; views assigned policies; downloads invoices; monitors commission earnings (5% of active policy premiums).
Admin	Manages all users (activate/deactivate); monitors all policies and claims system-wide; manages the 11 insurance plan products; creates, edits, and deactivates plans; views system-wide dashboard statistics.
Claims Officer	Reviews all auto-assigned claims; requests additional documents when needed; approves claims with a payout amount; rejects claims with a mandatory reason; processes payments; closes resolved claims.

4. Database Entities

4.1 ApplicationUser

Extends ASP.NET Identity's IdentityUser. Adds custom fields for role-based identity.

Field	Type	Description
Id	string (GUID)	Inherited from IdentityUser; primary key
FullName	string	User's full name
Email	string	Login email address
Role	string	Admin / Agent / Customer / ClaimsOfficer
IsActive	bool	Controls whether the user can log in
CreatedAt	DateTime	Timestamp of account creation

4.2 PolicyProduct

Represents one of the 11 predefined insurance plan products managed by Admin.

Field	Type	Description
PolicyProductId	int	Primary key
PolicyName	string	Plan name (e.g., Gold Single Trip)
PolicyType	string	Single Trip / Multi-Trip / Family / Student
PlanTier	string	Silver / Gold / Platinum
CoverageLimit	decimal	Maximum insured amount in USD
ClaimLimit	decimal	Maximum claim payout (50% of coverage)
BasePremium	decimal	Base rate used in premium formula
Tenure	int	30 days or 365 days
DestinationZone	string	Coverage zone (Worldwide)
CoverageDetails	string	Comma-separated list of covered benefits
ExclusionDetails	string?	Comma-separated list of exclusions (nullable)
Status	PolicyProductStatus	Available / Unavailable

4.3 PolicyRequest

Captures a customer's application for a policy before it is approved and purchased.

Field	Type	Description
PolicyRequestId	int	Primary key

PolicyProductId	int	FK to PolicyProduct
CustomerId	string	FK to ApplicationUser (customer)
AgentId	string?	Auto-assigned agent with least active policies
Destination	string	Travel destination country
StartDate / EndDate	DateTime	Trip travel dates
TravellerName	string	Primary traveller's name
TravellerAge	int	Used for premium and risk calculation
PassportNumber	string	Traveller's passport number
KycType / KycNumber	string	PAN / Aadhaar / CKYC with regex validation
RiskScore	decimal	Total score 0–100, visible to agent only
RiskAgeScore	decimal	Age component of risk
RiskDestinationScore	decimal	Destination component of risk
RiskDurationScore	decimal	Duration component of risk
RiskTierScore	decimal	Plan tier component of risk
RiskLevel	string	Low / Medium / High classification
Status	string	Pending / Approved / Rejected / Purchased
CalculatedPremium	decimal	Premium stored at submission time
RejectionReason	string?	Mandatory reason if agent rejects
AgentNotes	string?	Optional notes from reviewing agent
Documents	List<PolicyRequestDocument>	KYC, passport, and optional documents

4.4 Policy

Created when a customer pays for an approved PolicyRequest. Represents an active insurance policy.

Field	Type	Description
PolicyId	int	Primary key
PolicyNumber	string	Auto-generated: POL-YYYYMMDD-XXXX
CustomerId	string	FK to ApplicationUser
AgentId	string?	Agent inherited from the policy request
PolicyProductId	int	FK to PolicyProduct
TravellerName / Age	string / int	Traveller details from request

PassportNumber	string	From approved request
Destination	string	Travel destination
KycType / KycNumber	string	KYC details from request
PremiumAmount	decimal	Final premium paid by customer
StartDate / EndDate	DateTime	Policy validity period
Status	PolicyStatus	PendingPayment / Active / Expired / Cancelled

4.5 Claim

Submitted by a customer against an active policy. Managed by an auto-assigned Claims Officer.

Field	Type	Description
ClaimId	int	Primary key
PolicyId	int	FK to Policy
CustomerId	string	FK to ApplicationUser
ClaimsOfficerId	string?	Auto-assigned officer with least active claims
ClaimType	string	Type of claim (e.g., Medical, Baggage)
Description	string	Customer description of the incident
ClaimedAmount	decimal	Set to 0 on submission; officer decides payout
ApprovedAmount	decimal?	Payout amount entered by Claims Officer
Status	ClaimStatus	Submitted / UnderReview / PendingDocuments / Approved / Rejected / PaymentProcessed / Closed
RejectionReason	string?	Mandatory if officer rejects
SubmittedAt	DateTime	Claim creation timestamp
ReviewedAt	DateTime?	Officer review timestamp
Documents	List<ClaimDocument>	Uploaded supporting evidence

5. DTOs Used in the Application

Data Transfer Objects (DTOs) are used for all API communication. Domain entities are never exposed directly. DTOs are organized into request DTOs (incoming from the client) and response DTOs (outgoing to the client).

DTO Name	Direction	Purpose
RegisterRequestDto	Request	Customer registration: full name, email, password
LoginRequestDto	Request	Login: email and password

LoginResponseDto	Response	Returns JWT token and role after successful login
PolicyRequestSubmitDto	Request	Policy request form with trip details, KYC, and document files
PolicyRequestResponseDto	Response	Request details returned to customer (excludes risk score)
AgentRequestDetailDto	Response	Full request detail including risk score and breakdown (agent only)
PayRequestDto	Request	Contains PolicyRequestId for triggering payment
PolicyResponseDto	Response	Policy details for customer My Policies view
ClaimSubmitDto	Request	Claim type, description, and document uploads
ClaimResponseDto	Response	Full claim detail for customer and officer views
AgentReviewDto	Request	Agent decision: IsApproved, RejectionReason, AgentNotes
OfficerReviewDto	Request	Officer decision: IsApproved, RejectionReason, ApprovedAmount
AdminUserDto	Response	User listing with assigned agent info for Admin view
PolicyProductCreateDto	Request	Admin creates or edits an insurance plan product
DashboardDto	Response	Aggregated statistics for Admin and Agent dashboard views
NotificationDto	Response	Notification message with timestamp and read status

6. Backend Controllers and Endpoints

6.1 AuthController — /api/Auth

Method	Endpoint	Auth	Description
POST	/api/Auth/register	None	Register a new customer account
POST	/api/Auth/login	None	Authenticate and receive JWT token

6.2 PolicyRequestController — /api/PolicyRequest

Method	Endpoint	Auth	Description
POST	/api/PolicyRequest	Customer	Submit new policy request with documents (multipart/form-data)
GET	/api/PolicyRequest/my-requests	Customer	Retrieve all policy requests submitted by the logged-in customer

6.3 PolicyController — /api/Policy

Method	Endpoint	Auth	Description
GET	/api/Policy/products	None	Browse all available insurance plan products
GET	/api/Policy/my-policies	Customer	View all policies owned by the logged-in customer
POST	/api/Policy/pay-request	Customer	Pay for an approved policy request to activate the policy

6.4 AgentController — /api/Agent

Method	Endpoint	Auth	Description
GET	/api/Agent/dashboard	Agent	Dashboard statistics: total assigned, active, pending, expired
GET	/api/Agent/policies	Agent	View all policies assigned to the logged-in agent
GET	/api/Agent/policies/{id}	Agent	View full details of a specific policy including claims
GET	/api/Agent/policy-requests	Agent	View all policy requests assigned to the agent with risk badges
GET	/api/Agent/policy-requests/{id}	Agent	View request detail with risk score and breakdown
PUT	/api/Agent/policy-requests/{id}/review	Agent	Approve or reject a policy request

6.5 ClaimsOfficerController — /api/ClaimsOfficer

Method	Endpoint	Auth	Description
GET	/api/ClaimsOfficer/assigned	Officer	View all claims auto-assigned to the logged-in officer
PUT	/api/ClaimsOfficer/{id}/review	Officer	Approve or reject a claim
PUT	/api/ClaimsOfficer/{id}/request-docs	Officer	Request additional documents from the customer
PUT	/api/ClaimsOfficer/{id}/process-payment	Officer	Process payment for an approved claim
PUT	/api/ClaimsOfficer/{id}/close	Officer	Close a fully processed claim

6.6 AdminController — /api/Admin

Method	Endpoint	Auth	Description
GET	/api/Admin/dashboard	Admin	System-wide statistics across all users, policies, and claims
GET	/api/Admin/users	Admin	View all users grouped by role with assigned agent info
PUT	/api/Admin/users/{id}/toggle-status	Admin	Activate or deactivate a user account
GET	/api/Admin/policies	Admin	View all policies system-wide
GET	/api/Admin/claims	Admin	View all claims system-wide
GET	/api/Admin/policy-products	Admin	View all insurance plan products
POST	/api/Admin/policy-products	Admin	Create a new insurance plan product
PUT	/api/Admin/policy-products/{id}	Admin	Edit an existing plan product

7. Backend Services and Functions

7.1 PolicyRequestService

- SubmitPolicyRequestAsync: Validates KYC format using regex, calculates premium, computes risk score, auto-assigns the agent with the fewest active policies, saves the request and uploaded documents, and sends a notification.
- GetMyRequestsAsync: Retrieves all policy requests for the authenticated customer, mapped to response DTOs excluding risk score fields.

7.2 PremiumCalculationService

- CalculatePremium: Applies the formula — BasePremium × (TripDays / 30) × AgeLoading for Single Trip and Family plans. For Multi-Trip and Student plans, applies BasePremium × AgeLoading (fixed annual). Family plans additionally multiply by the member multiplier based on total insured members (2 to 6 members).
- GetAgeLoading: Returns 1.0 for age ≤ 40, 1.1 for age 41–60, and 1.3 for age above 60.
- GetMemberMultiplier: Returns multipliers from 1.5 (2 members) to 2.5 (6 members) for family plan calculations.

7.3 RiskAssessmentService

- CalculateRiskScore: Computes a score from 0 to 100 by summing four components — Age Risk (0–30), Destination Risk (5–30), Duration Risk (0–20), and Plan Tier Risk (0–20). The resulting score is stored in the database and classified as Low (0–30), Medium (31–60), or High (61–100).
- The risk score is never included in any customer-facing response DTO.

7.4 AgentService

- GetAssignedRequestsAsync: Returns all policy requests assigned to the agent, including risk score and factor breakdown.
- ReviewPolicyRequestAsync: Processes agent decisions — updates status to Approved or Rejected, stores notes or rejection reason, and triggers a customer notification.
- GetDashboardAsync: Returns counts of total assigned, active, pending, and expired policies for the agent's dashboard.
- GetCommissionAsync: Calculates 5% of all active policy premiums assigned to the agent.

7.5 PolicyService

- PayForRequestAsync: Verifies the request belongs to the authenticated customer and is in Approved status, creates a Policy record with status Active, generates a unique PolicyNumber (POL-YYYYMMDD-XXXX), updates the request status to Purchased, and sends a notification.
- GetMyPoliciesAsync: Returns all policies for the authenticated customer mapped to response DTOs.

7.6 ClaimService

- SubmitClaimAsync: Creates a new claim against an active policy, sets ClaimedAmount to zero, saves uploaded documents to wwwroot/claims/{claimId}/, and auto-assigns the Claims Officer with the fewest active claims. Status is set to Submitted, then immediately transitioned to UnderReview.
- ReviewClaimAsync: Processes officer decisions — approve (with payout amount), reject (with mandatory reason), request additional documents, process payment, or close the claim.
- GetAssignedClaimsAsync: Returns all claims assigned to the logged-in Claims Officer, filterable by status.

7.7 NotificationService

- CreateNotificationAsync: Creates a notification record whenever a policy request or claim status changes. Notification messages are role-appropriate and stored for display in the customer portal.
- GetMyNotificationsAsync: Returns all unread and read notifications for the authenticated user in reverse chronological order.

8. Frontend Structure

The Angular application is organized by feature and role. Each role has its own route group and layout component. All components are standalone — no NgModules are used.

8.1 Application Route Groups

Route Prefix	Role	Key Pages
/auth	None	Login, Register
/browse	Public	Browse Plans, Plan Detail, Premium Calculator
/customer	Customer	My Requests, My Policies, Submit Claim, My Claims, Notifications, Profile
/agent	Agent	Dashboard, Policy Requests, Request Detail, My Policies, Policy Detail
/admin	Admin	Dashboard, Users, All Policies, All Claims, Plan Products, Create/Edit Plan
/officer	Claims Officer	Assigned Claims, Claim Detail, Process Actions

8.2 Core Services (Angular)

- AuthService: Manages login, registration, JWT storage, token decoding, and logout.
- PolicyRequestService: Handles form submission with multipart/form-data and fetches customer requests.
- PlanService: Fetches available plan products, filters by type and zone, and calculates live premiums for browse page.
- ClaimService: Manages claim submission, document uploads, and claim status polling.
- AgentService: Provides agents with request lists, request details with risk data, and review submission.
- AdminService: Fetches system-wide data, manages users, and handles plan product CRUD operations.
- OfficerService: Fetches and filters assigned claims, submits officer decisions and payment actions.
- NotificationService: Polls for new notifications and displays them in the customer header.

8.3 Angular Guards

- AuthGuard: Reads the decoded role from the JWT stored in localStorage. Redirects unauthenticated users to /auth/login. Redirects authorized users who access a wrong role's route.
- NoAuthGuard: Prevents already-authenticated users from accessing /auth/login and /auth/register, redirecting them to their role's dashboard.

9. Frontend Functions and Logic

9.1 Browse Plans Page

- User selects a plan type (Individual, Family, Frequent Flier, Student), destination country, travel dates, and traveller age.
- Angular Signals reactively update the displayed plan cards whenever any filter value changes.
- Each plan card displays the calculated premium for the entered trip parameters, computed client-side using the same formula as the backend.
- Clicking Request Policy opens a modal pre-filled with the entered trip details.

9.2 Policy Request Modal

- Customer completes remaining fields: traveller name, passport number, KYC type selection, and KYC number.
- Frontend performs regex pre-validation for PAN (format: ABCDE1234F), Aadhaar (12 digits), and CKYC (14 digits).
- KYC document and passport copy are required uploads. A third supporting document is optional.
- Form is submitted as multipart/form-data to the backend via POST /api/PolicyRequest.

9.3 My Requests Page

- Lists all policy requests with their current status displayed as a colored badge (Pending, Approved, Rejected, Purchased).
- Polls the backend every 30 seconds to refresh status without requiring a page reload.
- Approved requests display a Buy Now button that navigates to the payment page.

9.4 Payment Page

- Displays the pre-calculated premium from the approved request.
- On confirmation, calls POST /api/Policy/pay-request with the request ID.
- On success, redirects to /customer/my-policies.

9.5 Agent Request Detail Page

- Displays full request details including risk score, per-factor breakdown, risk level badge (green/yellow/red), and uploaded document links.
- Agent enters optional notes and selects Approve or Reject.
- Rejection requires a mandatory reason before the form can be submitted.

9.6 Claims Officer Portal

- Claim list is filterable by status tab: All, UnderReview, PendingDocuments, Approved, Rejected, Closed.
- Claim detail page shows customer information, policy details, claim description, and all uploaded documents.
- Officer actions are rendered as conditional buttons based on the current claim status.

10. Policy Request and Policy Buying Flow

Phase 1 — User Registration

- One Admin account is pre-seeded in the database on application startup.
- Three Agents and one Claims Officer are seeded for the development environment.
- Eleven insurance plan products are seeded if the PolicyProducts table is empty.
- Customers self-register via the registration page and are immediately assigned the Customer role.

Phase 2 — Browse Plans

- Customer navigates to the Browse Plans page and selects plan type, destination, travel dates, and age.
- The system filters and displays matching plan cards with live premium calculations.
- Customer selects a plan and clicks Request Policy to open the submission modal.

Phase 3 — Policy Request Submission

- Customer completes the request form with traveller details, KYC information, and document uploads.
- Backend validates KYC format, calculates the premium, and computes the risk score.
- System auto-assigns the agent with the fewest currently active policies.
- PolicyRequest is created with status Pending. Documents are saved to wwwroot/policy-requests/{requestId}/.
- Customer can view the request under My Requests, with status polling every 30 seconds.

Phase 4 — Agent Review

- Agent sees the new request in their portal with a Pending badge.
- Agent opens the request detail page and reviews the risk score, factor breakdown, and uploaded documents.
- Agent may add optional notes and either approves or rejects the request.
- Rejection requires a mandatory reason. Approval sends a notification to the customer.

Phase 5 — Policy Purchase

- Customer sees the Buy Now button on the approved request in My Requests.
- Payment page loads the pre-calculated premium. Customer confirms payment.
- Backend verifies the request is in Approved status and belongs to the authenticated customer.
- A Policy is created with status Active. The PolicyRequest status is updated to Purchased to prevent duplicate payment.
- Customer is redirected to My Policies.

11. Claim Processing Flow

Phase 6 — Claim Submission

- Customer selects an Active policy from My Policies and fills the claim form with claim type and incident description.
- Customer uploads supporting documents. System creates the claim with status Submitted.
- System auto-assigns the Claims Officer with the fewest currently active claims.
- Claim status transitions immediately to UnderReview after assignment.

Phase 7 — Claims Officer Review

- Claims Officer views all assigned claims in their portal, filterable by status.
- Officer opens the claim detail page, reviews documents, and takes one of the following actions:
  - Request Additional Documents: Status changes to PendingDocuments. Customer uploads more evidence, which returns the status to UnderReview.
  - Approve Claim: Officer enters the approved payout amount. Status changes to Approved.
  - Reject Claim: Officer provides a mandatory rejection reason. Status changes to Rejected.
- For approved claims, Officer processes the payment, updating status to PaymentProcessed.
- Officer closes the claim, updating the final status to Closed.

11.1 Claim Status Lifecycle

From Status	To Status	Trigger
(New)	Submitted	Customer submits claim
Submitted	UnderReview	System auto-assigns Claims Officer
UnderReview	PendingDocuments	Officer requests additional documents
PendingDocuments	UnderReview	Customer uploads requested documents
UnderReview	Approved	Officer approves with payout amount
UnderReview	Rejected	Officer rejects with mandatory reason
Approved	PaymentProcessed	Officer processes payment
PaymentProcessed	Closed	Officer closes the claim

11.2 Policy Request Status Lifecycle

From Status	To Status	Trigger
(New)	Pending	Customer submits request
Pending	Approved	Agent approves the request

Pending	Rejected	Agent rejects with mandatory reason
Approved	Purchased	Customer completes payment

12. Third-Party Integrations

Talk & Travel is designed as a self-contained system and does not integrate with external third-party services in the initial version. All features — authentication, premium calculation, risk scoring, document storage, and notification delivery — are implemented natively within the application stack.

The following framework-level libraries and packages are used:

Library / Package	Purpose
Microsoft.AspNetCore.Identity	User management, password hashing, and role assignment
Microsoft.AspNetCore.Authentication.JwtBearer	JWT Bearer token validation middleware
Microsoft.EntityFrameworkCore + SqlServer	ORM for database access and EF Core migrations
System.IdentityModel.Tokens.Jwt	JWT token generation on login
xUnit + Moq	Unit and integration testing framework with mock support
Microsoft.EntityFrameworkCore.InMemory	In-memory database for repository tests without SQL Server
@angular/common/http	Angular HTTP client for REST API communication
TailwindCSS	Utility-first CSS framework for frontend styling

Payment processing in this version is simulated — the system records a payment confirmation without connecting to a real payment gateway. A future version may integrate with Razorpay or Stripe for live payment processing.

13. Security and Authentication

13.1 Authentication Mechanism

The system uses ASP.NET Identity combined with JWT Bearer tokens for authentication. On successful login, the backend issues a signed JWT containing the user's ID, email, and role. This token is stored client-side and sent in the Authorization header as a Bearer token on every subsequent API request.

13.2 Authorization

- Every API endpoint is decorated with [Authorize(Roles = "...")], ensuring only users with the correct role can access that endpoint.
- The Angular Auth Guard reads the role from the decoded JWT on every route navigation and redirects unauthorized users.
- The No-Auth Guard prevents logged-in users from accessing the login or registration pages.

13.3 Password Security

- Passwords are hashed using ASP.NET Identity's built-in password hasher, which applies bcrypt-based hashing with a salt.
- Plain text passwords are never stored or transmitted.

13.4 Data Protection

- Risk scores are never included in customer-facing response DTOs. They are stored in the database and exposed only through agent-authenticated endpoints.
- Document URLs are served as static files. File paths are not guessable as they are stored under request-specific subdirectories.
- CORS is configured to accept requests only from the Angular development server origin.

13.5 KYC Validation

- PAN: Must match the pattern ABCDE1234F — five uppercase letters, four digits, one uppercase letter.
- Aadhaar: Must be exactly 12 numeric digits.
- CKYC: Must be exactly 14 numeric digits.

Validation is performed on the backend via regex. The frontend also performs pre-validation to improve the user experience before submission.

13.6 Document Storage Security

- Policy request documents are saved to wwwroot/policy-requests/{requestId}/.
- Claim documents are saved to wwwroot/claims/{claimId}/.
- Accepted file types are limited to PDF, JPG, JPEG, and PNG.
- Maximum file size is 5 MB per file.
- File metadata (original name, stored name, upload time) is persisted in the PolicyRequestDocument and ClaimDocument entities.

14. Features Implemented in the System

14.1 Insurance Plan Management

- Eleven predefined plans across four plan types (Single Trip, Multi-Trip, Family, Student) and three tiers (Silver, Gold, Platinum).
- Each tier inherits all benefits of the tier below, with Platinum offering the most comprehensive coverage.

- Admin can create, edit, and toggle availability of plan products at any time.

14.2 Premium Calculation Engine

- Single Trip and Family plans: Premium = BasePremium × (TripDays / 30) × AgeLoading (× MemberMultiplier for Family).
- Multi-Trip and Student plans: Premium = BasePremium × AgeLoading (fixed annual, not prorated).
- Age loading: 1.0 for age ≤ 40, 1.1 for age 41–60, 1.3 for age above 60.
- Member multiplier for Family plans ranges from 1.5 (2 members) to 2.5 (6 members).

14.3 Risk Assessment Engine

- Risk score computed on every policy request submission using four factors: Age Risk (0–30), Destination Risk (5–30), Duration Risk (0–20), and Plan Tier Risk (0–20).
- Score classified as Low (0–30), Medium (31–60), or High (61–100).
- Agent sees a color-coded badge and per-factor breakdown. Customer never sees the risk data.

14.4 Agent Auto-Assignment

- On policy request submission, the system queries all active agents and selects the one with the fewest currently active policies.
- This ensures workload is distributed evenly across agents.

14.5 Claims Officer Auto-Assignment

- On claim submission, the system selects the Claims Officer with the fewest active (non-closed, non-rejected) claims.
- This mirrors the agent assignment logic and ensures balanced claims workloads.

14.6 Structured Claim Lifecycle

- Seven distinct claim statuses: Submitted, UnderReview, PendingDocuments, Approved, Rejected, PaymentProcessed, Closed.
- Each transition is triggered by a specific role action and is enforced in the backend service layer.
- Officers cannot skip steps — payment can only be processed after approval, and closure requires payment.

14.7 Notification System

- In-app notifications are created for every status change on policy requests and claims.
- Notifications are visible in the customer portal header.
- Stored with a timestamp and read/unread state.

14.8 Document Upload and Management

- Policy requests accept KYC document, passport copy (required), and one optional supporting document.

- Claims accept multiple supporting evidence files.
- All files are stored on the server file system and accessible through static file links.

14.9 Agent Commission Tracking

- Each agent can view their total commission earned, calculated as 5% of the total premium amount across all active policies assigned to them.
- Commission is displayed on the agent dashboard.

14.10 Role-Based Dashboards

- Customer: My Requests, My Policies, My Claims, Notifications.
- Agent: Total assigned, pending, active, and expired policy counts; commission summary.
- Admin: System-wide totals — users, policies, claims, and plan products.
- Claims Officer: All assigned claims with status filter tabs.

15. Summary

Talk & Travel is a comprehensive, production-ready travel insurance management system built on a modern, scalable technology stack. The system implements end-to-end digital insurance workflows — from plan browsing and policy request submission through agent verification, customer payment, claim submission, and claim resolution — within a single unified platform.

The backend's Clean Architecture ensures separation of concerns, testability, and maintainability. Each of the four layers — Domain, Application, Infrastructure, and API — has a single, well-defined responsibility with no circular dependencies. The Angular frontend provides a responsive, role-aware user interface using standalone components, Angular Signals, and TailwindCSS.

Key differentiators of the system include its formula-driven premium engine, automated risk scoring model, intelligent workload-balanced auto-assignment for both agents and claims officers, and a fully structured claim lifecycle enforced at the service layer. Security is enforced at every level — JWT Bearer tokens, role-based endpoint authorization, Angular route guards, bcrypt password hashing, and KYC regex validation.

The system is designed for extensibility. Future enhancements may include live payment gateway integration (Razorpay or Stripe), email notification delivery, PDF invoice generation, advanced analytics dashboards, and policy renewal workflows.